

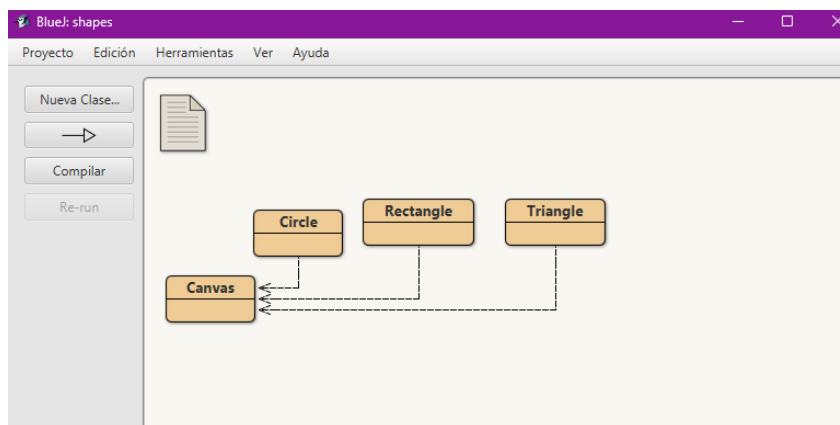
Responsables: José Luis García, Kevin David Quezada

PARTE I. SHAPES

A Conociendo el proyecto.

1. El proyecto shapes es una versión modificada de un recurso ofrecido por BlueJ. Para trabajar con él, bajen shapes.zip y ábralo en BlueJ1. Capturen la pantalla.

R/=



2. El diagrama de clases permite visualizar las clases de un artefacto software y las relaciones entre ellas. Considerando el diagrama de clases de shapes:
 - a. ¿Qué clases ofrece?
R/= Ofrece las clases Canvas, Circle, Rectangle, Triangle.
 - b. ¿Qué relaciones existen entre ellas?
R/= Triangle, Rectangle y Circle usan y dependen de Canvas.
3. La documentación presenta las clases del proyecto y, en este caso, la especificación de sus componentes públicos. Genere la documentación. De acuerdo con la documentación:
 - a. ¿Qué clases tiene el paquete shapes?
R/= Canvas, Circle, Rectangle, Triangle.

Classes	
Class	Description
Canvas	Canvas is a class to allow for simple graphical drawing on a canvas.
Circle	A circle that can be manipulated and that draws itself on a canvas.
Rectangle	A rectangle that can be manipulated and that draws itself on a canvas.
Triangle	A triangle that can be manipulated and that draws itself on a canvas.

- b. ¿Qué atributos tiene la clase Triangle?
R/= Public: VERTICES

Private: height, width, xPosition, yPosition, color, isVisible

c. ¿Cuántos métodos ofrece la clase Triangle?

R/= Sin contar el constructor, 12

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
void	changeColor(String[Ⓜ] newColor)	Change the color.
void	changeSize(int newHeight, int newWidth)	Change the size to the new size
void	makeInvisible()	Make this triangle invisible.
void	makeVisible()	Make this triangle visible.
void	moveDown()	Move the triangle a few pixels down.
void	moveHorizontal(int distance)	Move the triangle horizontally.
void	moveLeft()	Move the triangle a few pixels to the left.
void	moveRight()	Move the triangle a few pixels to the right.
void	moveUp()	Move the triangle a few pixels up.
void	moveVertical(int distance)	Move the triangle vertically.
void	slowMoveHorizontal(int distance)	Slowly move the triangle horizontally.
void	slowMoveVertical(int distance)	Slowly move the triangle vertically.
Methods inherited from class java.lang.Object[Ⓜ]		
clone[Ⓜ] , equals[Ⓜ] , getClass[Ⓜ] , hashCode[Ⓜ] , notify[Ⓜ] , notifyAll[Ⓜ] , toString[Ⓜ] , wait[Ⓜ] , wait[Ⓜ] , wait[Ⓜ]		

d. ¿Qué atributos determinan el tamaño de un Triangle?

R/= width y height.

e. ¿Cuáles métodos ofrece la clase Triangle para cambiar su tamaño?

R/=

void	changeSize(int newHeight, int newWidth)	Change the size to the new size
------	---	---------------------------------

4. En el código fuente de cada clase está el detalle de la implementación. Revisen el código de la clase Triangle. Con respecto a los atributos:

a. ¿Cuántos atributos realmente tiene?

R/= 6.

Public: VERTICES. Private: height, width, xPosition, yPosition, color, isVisible.

```

12 public static int VERTICES=3;
13
14 private int height;
15 private int width;
16 private int xPosition;
17 private int yPosition;
18 private String color;
19 private boolean isVisible;
20

```

b. ¿Quiénes pueden usar los atributos públicos?

R/= VERTICES, lo puede usar en este caso todas las clases del mismo paquete shapes: Canvas, Rectangle, Circle.

Con respecto a los métodos:

c. ¿Cuántos métodos tiene en total?

R/= Con el constructor, 15

d. ¿Quiénes usan los métodos privados?

R/= draw y erase solo pueden ser usados por los métodos dentro de Triangle,

5. Desde el editor, consulte la documentación de Triangle.

a. Capture la pantalla.

R/=

Class Triangle

java.lang.Object²
Triangle

public class Triangle
extends Object²

A triangle that can be manipulated and that draws itself on a canvas.

Version:
1.0 (15 July 2000)

Author:
Michael Kolling and David J. Barnes

Field Summary

Fields

Modifier and Type	Field	Description
static int	VERTICES	

Constructor Summary

Constructors

Constructor	Description
Triangle()	Create a new triangle at default position with default color.

Method Summary

b. ¿Qué no se ve en la documentación?

R/= Los atributos y métodos privados. El código de cada método.

c. ¿por qué debe ser así?

R/= Porque la documentación debe mostrar el contrato público de la clase, nunca el estado interno ni la implementación.

6. ¿Cuál dirían es el propósito del proyecto shapes?

R/= Introducir de forma práctica los conceptos básicos de POO

B Manipulando objetos.

1. Creen un objeto de cada una de las clases que lo permitan y háganlos visible.

a. ¿Cuántas clases tiene el proyecto?

R/= Sigue habiendo cuatro clases.

b. ¿Cuántos objetos crearon?

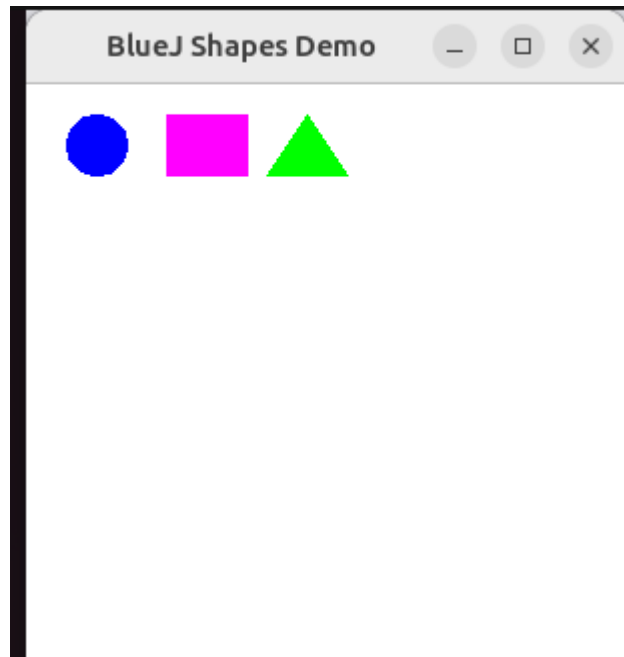
R/= Tres

c. ¿Qué clase se crea de forma diferente? ¿Por qué?

R/= Canvas es la única que se crea diferente

d. Capturen la pantalla.

R/=



2. Inspeccionen los creadores de cada una de las clases.

a. ¿Cuál es la principal diferencia entre ellos?

R/= Las clases, Triangle y Rectangle son creadas por: Michael Kolling y David J. Barnes. La clase Canvas por: Bruce Quig y Michael Kolling (mik).

b. ¿Qué se busca con la clase que tiene el creador diferente?

R/= Garantizar que exista una única instancia de Canvas en todo el programa.

3. Construyan, con shapes sin escribir código, una propuesta de la imagen del logo de su navegador favorito.

a. ¿Cuántas y cuáles clases se necesitan?

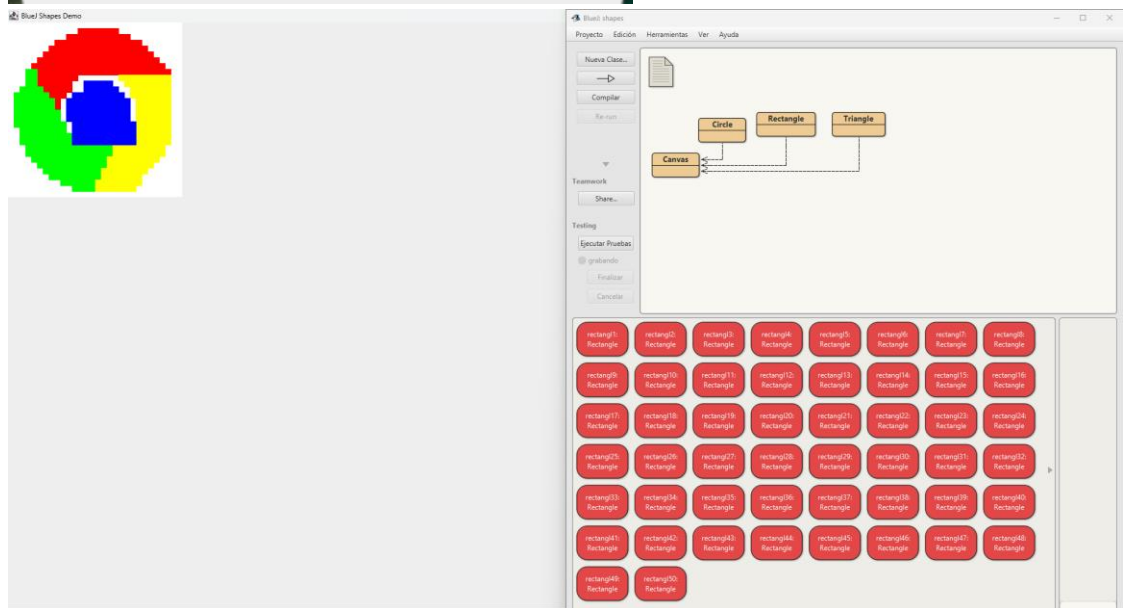
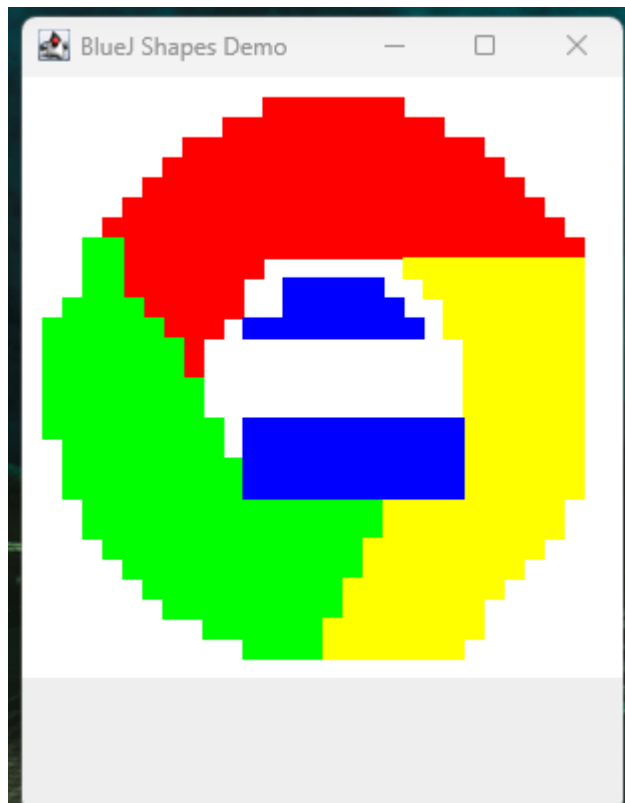
R/= Solo Rectangle

b. ¿Cuántos objetos se usan en total?

R/= 50

c. Capturen la pantalla.

R/=



- d. Incluyan el logo original
R/=



C Detallando una clase y explorando sus objetos.

1. En el código de la clase Triangle, revise el atributo VERTICES.
 - a. ¿Qué significa que sea public?
R/= Que es accesible desde cualquier otra clase: se puede leer (y modificar) desde fuera como Triangle.VERTICES. No esta encapsulado.
 - b. ¿Qué significa que sea static?
R/= Que pertenece a la clase y no a cada objeto: existe una sola copia de VERTICES compartida por todos los triángulos, sin importar cuantos objetos se creen.
 - c. ¿Debería ser final? ¿Por qué?
R/= Sí. Es una constante: todo triangulo tiene exactamente 3 vértices por definición, no tiene sentido que cambie. final impide que se reasigne, protegiendo la invariante.
 - d. Realicen los cambios necesarios.
R/= `public static final int VERTICES = 3;`
2. En el código de la clase Triangle revisen el detalle del tipo del atributo height.
 - a. ¿Qué se está indicando al decir que es int?
R/= Que height almacena números enteros de 32 bits (rango de -2^{31} a $2^{31}-1$). Aquí representa la altura en pixeles.
 - b. Si fuera byte, ¿cuál sería el área más grande posible?
R/= byte tiene rango -128 a 127. Con valores positivos, el maximo es 127. El area del triangulo se calcula como $(width \times height) / 2$. Si tanto width como height fueran byte: $area\ max = 127 \times 127 / 2 = 8064$ (division entera).
 - c. y ¿si fuera long?
R/= long tiene rango de -2^{63} a $2^{63}-1$: el area maxima posible es enorme (del orden de 2^{126}), sin limite practico para una figura en pantalla. Pero desperdicia memoria (8 bytes por atributo en vez de 4).
 - d. ¿qué restricción adicional debería tener este atributo?
R/= Que el valor debe ser no negativo (≥ 0): un triangulo no puede tener altura (ni ancho) negativo.
 - e. Refactoricen el código considerando (d).

R/=

3. Si creamos 100 triángulos,

- a. ¿cuántos VERTICES y cuántos height tendríamos?

R/= Tendríamos 100 atributos height (uno por objeto) y 1 solo VERTICES.

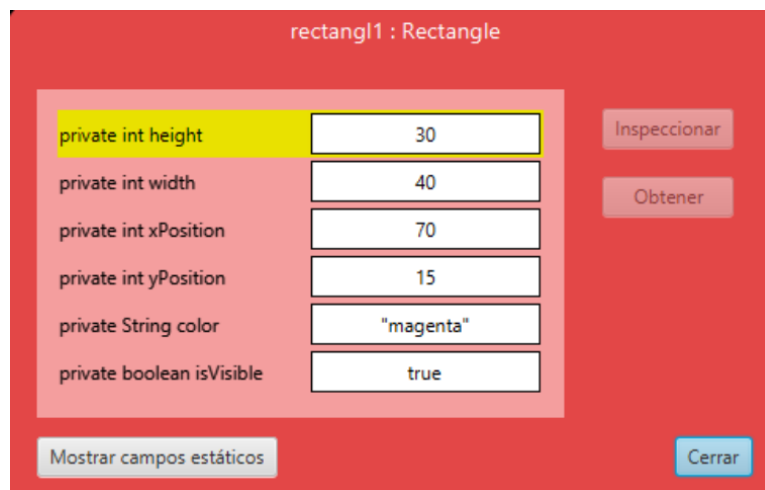
- b. Justifique.

R/= height es un atributo de instancia: cada objeto tiene su propia copia en memoria. VERTICES es static: hay una única copia a nivel de clase, compartida por todos los objetos.

4. Inspeccionen el estado del objeto: Triangle.

- a. Capturen la pantalla.

R/=



- b. ¿Cuál es el valor de height inicial?

R/= 30 (valor asignado en el constructor).

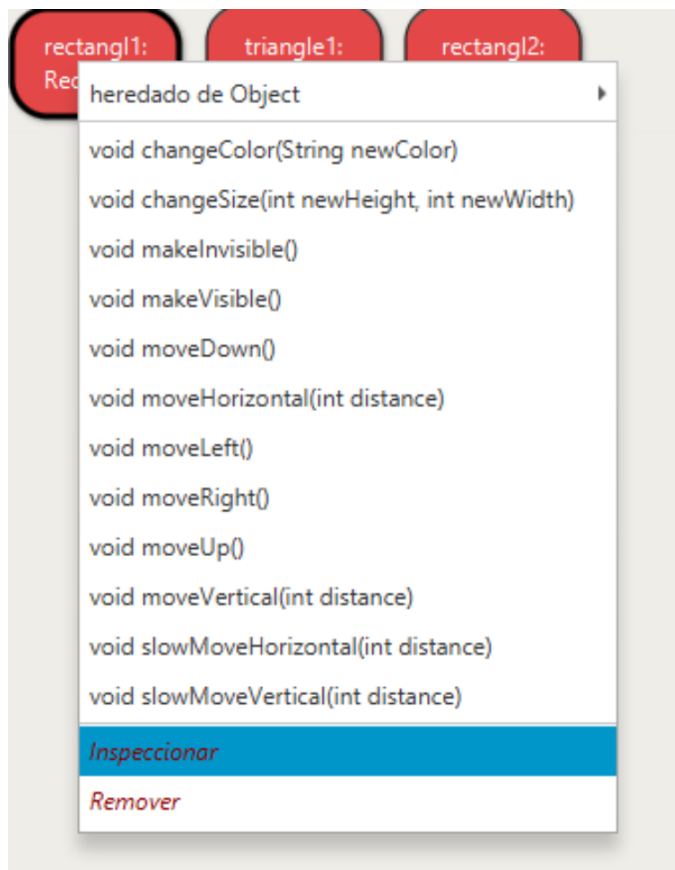
- c. ¿Cuál es el valor de VERTICES?

R/= 1 que sería el static.

5. Inspeccionen el comportamiento que ofrece el objeto :Triangle.

- a. Capturen la pantalla.

R/=



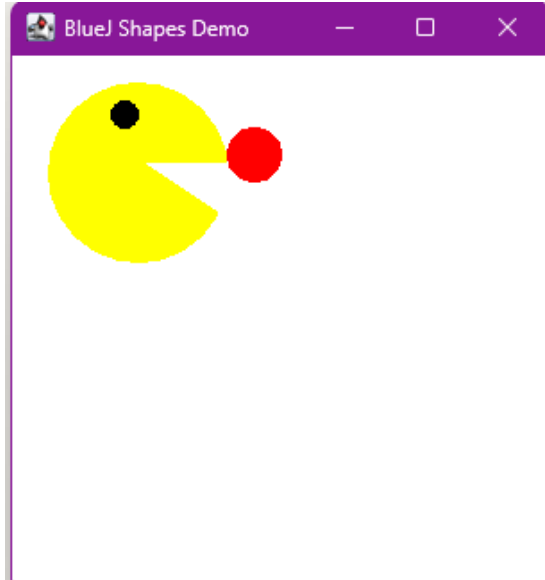
- b. ¿Por qué no aparecen todos los métodos que están en el código?
 R/= Porque el menú contextual del objeto muestra solo los métodos públicos. Los métodos privados (draw, erase) no se exponen al usuario del objeto: es el encapsulamiento en acción. La clase usa internamente esos métodos, pero quien manipula el objeto desde afuera solo ve la interfaz pública.

D Analizando y escribiendo código.

```
Circle face;
Circle eye;
Triangle mouthUp;
Triangle mouthDown;
//1
face = new Circle();
face.changeColor("yellow");
face.changeSize(100);
//2
eye = new Circle();
eye.changeColor("black");
eye.changeSize(15);
eye.moveVertical(10);
eye.moveHorizontal(35);
eye.makeVisible();
//3
```

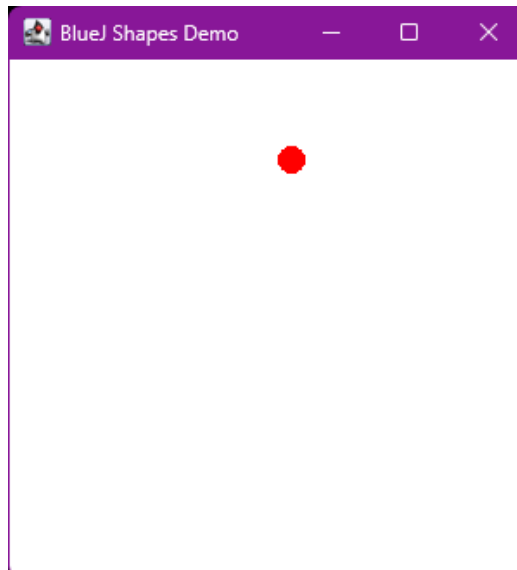
```
mouthUp = new Triangle();
mouthUp.changeColor("white");
mouthUp.changeSize(30, 90);
mouthUp.moveHorizontal(-20);
mouthUp.moveVertical(15);
mouthUp.makeVisible();
//4
mouthDown = new Triangle();
mouthDown.changeColor("white");
mouthDown.changeSize(-30, 90);
mouthDown.moveHorizontal(-20);
mouthDown.moveVertical(75);
mouthDown.makeVisible();
//5
Circle food=eye;
food.changeColor("red");
food.moveHorizontal(100);
food.moveVertical(25);
food.makeVisible();
//6
```


1. Lean el código anterior.
 - a. ¿Cuál creen que es la figura resultante?
R/= Una cara.
 - b. Píntenla.
R/=



2. Para cada punto señalado:
 - a. ¿cuántas variables existen?
R/=
Para 1: 4 variables.
Para 2: 4 variables.
Para 3: 4 variables.
Para 5: 4 variables.
Para 6: 5 variables.
 - b. ¿cuántos objetos existen?
R/=
Para 1: 0 objetos.
Para 2: 1 objeto.
Para 3: 2 objetos.
Para 4: 3 objetos.
Para 5: 4 objetos.
Para 6: 4 objetos. (food y eye apuntan al mismo Circle en memoria).
 - c. ¿cuántos objetos se ven?
Para 1: 0 visibles.
Para 2: 0 visibles.
Para 3: 1 visible.
Para 4: 2 visibles.
Para 5: 3 visibles.
Para 6: 3 visibles.

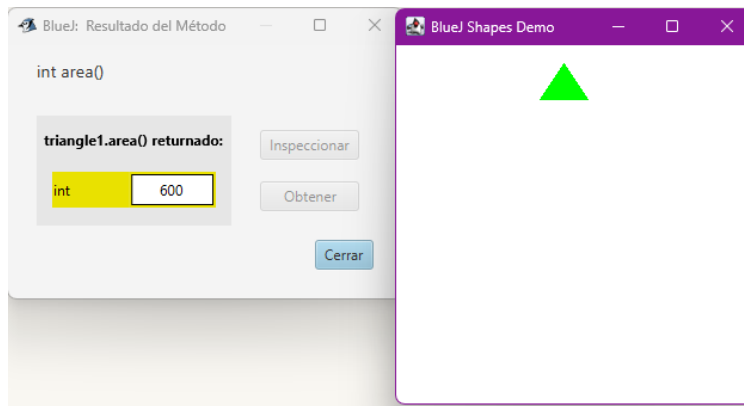
3. Habiliten la ventana de código en línea 7, escriban el código.
 - a. Capturen la pantalla.
 - b. R/=



4. Compare la figura pintada en 1. con la figura capturada en 3.:
 - a. ¿son iguales?
R/= No.
 - b. ¿por qué?
R/= El otro era un pacman comiendose una dot roja, y en este solo se vé la dot.
5.
 - a. ¿Qué errores hacen que la figura no quede bien pintada?
R/= Falta `face.makeVisible()`, que impide que se vea el cuerpo de Pacman, y como el color de los triángulos de la boca son blancos, se confunden con el cuerpo del fondo blanco de Canvas.
 - b. ¿Qué figura es?
R/= Pacman comiendose una dot roja.

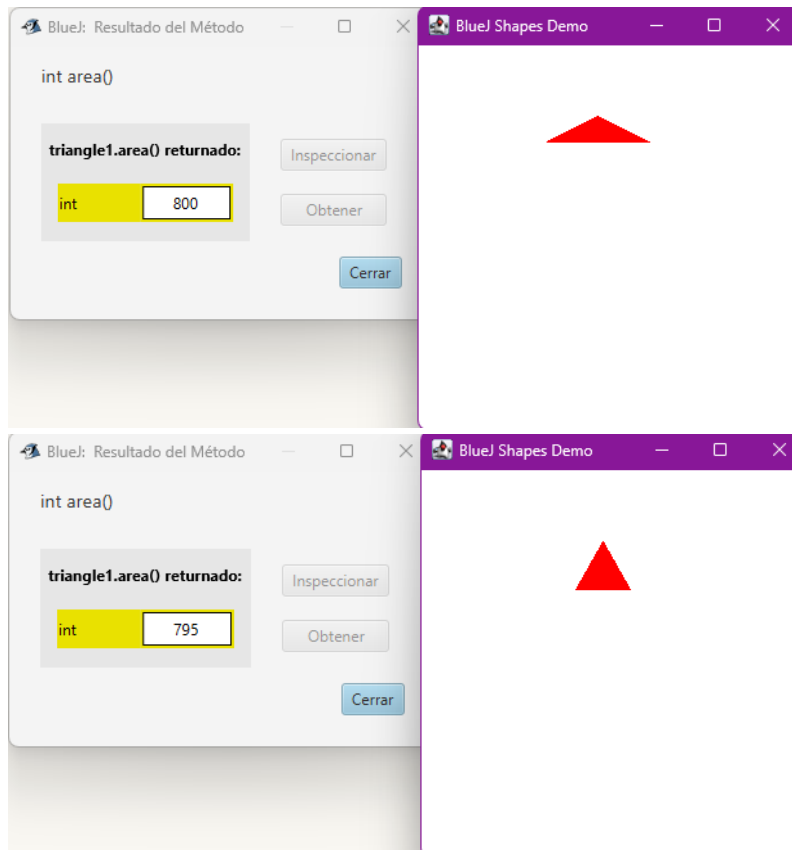
E Extendiendo una clase.

1. Desarrollen en `Triangle` el método `area()` (retorna el área). ¡Pruébenlo! Capturen una pantalla.
R/=



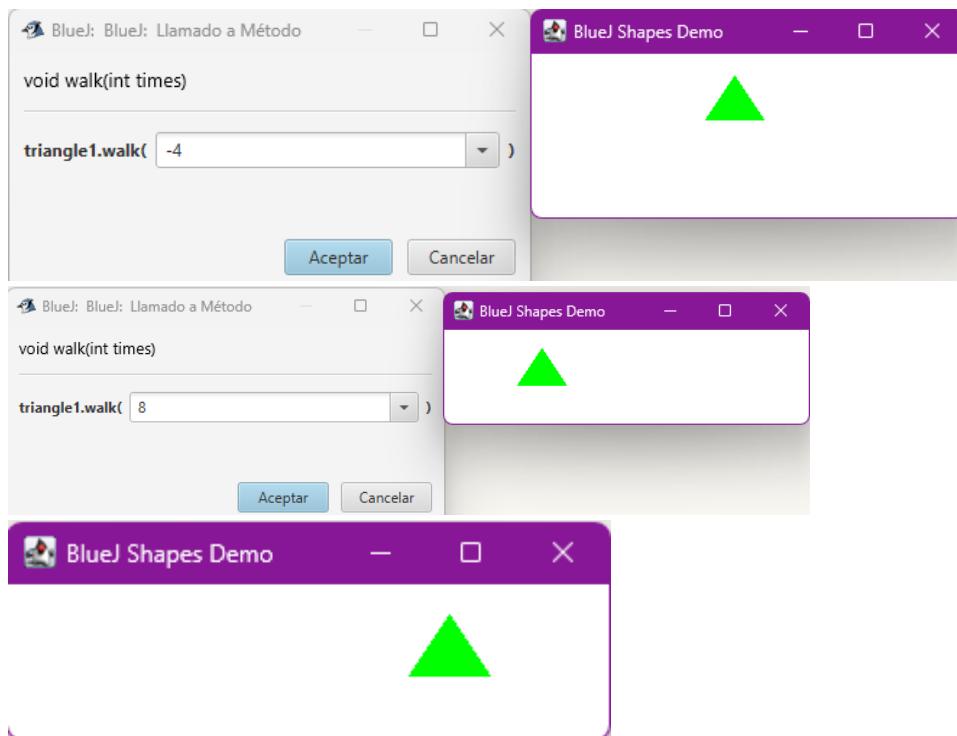
- Desarrollen en Triangle el método `equilateral()` (lo convierte en un triángulo equilátero de aproximada mente la misma área); Pruébenlo! Capturen dos pantallas.

R/=



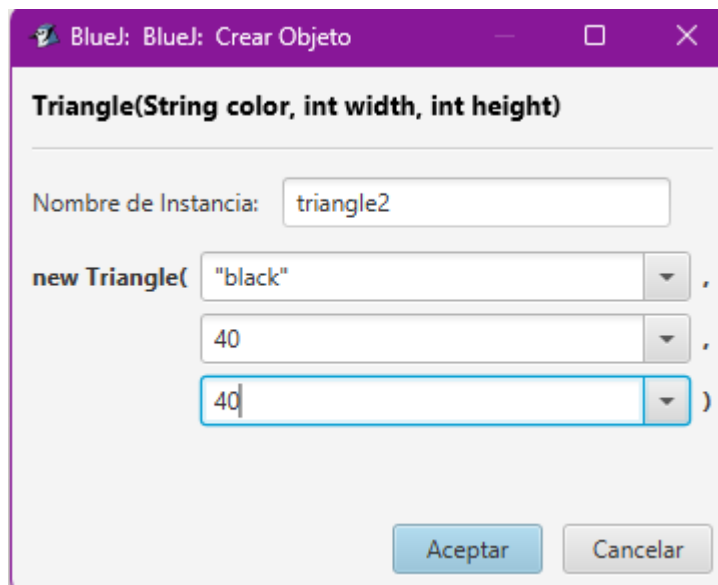
- Desarrollen en Triangle el método `walk(times: int)` (va moviendose cayendo hacia la derecha (positivo) o izquierda (negativo) `abs(times)` veces). ¡Pruébenlo! Capturen tres pantallas.

R/=



4. Desarrollen en Triangle un nuevo creador que permita crearlo dado su color, su ancho y su alto. ¡Pruébenlo! Capturen una pantalla.

R/=



5. Propongan un nuevo método para esta clase. Desarrollen y prueben el método.

R/=

```
public double perimeter(){
    double lado = Math.sqrt(Math.pow(width / 2.0, 2) + Math.pow(height, 2));
    return width + 2 * lado;
}
```

6. Generen nuevamente la documentación y revisen la información de estos nuevos métodos. Capturen la pantalla.

R/=

Classes	
Class	Description
Canvas	Canvas is a class to allow for simple graphical drawing on a canvas.
Circle	A circle that can be manipulated and that draws itself on a canvas.
Rectangle	A rectangle that can be manipulated and that draws itself on a canvas.
Robot	Representa un robot que se desplaza dentro de un laberinto (RobotMaze).
Triangle	A triangle that can be manipulated and that draws itself on a canvas.

PARTE II. De Python a Java ☒

En este punto vamos a usar y evaluar dos recursos de apoyo para la transición de Python a Java. Realicen la evaluación en las encuestas preparadas con ese objetivo:

1. El video
2. Los prompts.

PARTE III. RobotMaze

A Creando una nueva clase. Usando el paquete shapes.

1. Inicie la construcción únicamente con los atributos. Justifique su selección. Agregue pantallazo con los atributos.

R/=

Se eligieron xPosition, yPosition para la posición lógica del robot en el laberinto; direction para la orientación actual (inicia en 'N', según lo indicado); isVisible para controlar el estado de dibujo; y body, antenna, antennaTip como las 3 figuras de shapes que componen visualmente al robot. Se usa CELL_SIZE para traducir coordenadas lógicas del laberinto a posiciones de píxeles en el Canvas, dado que las clases de shapes no ofrecen una forma de fijar posición absoluta, solo desplazamiento relativo.

```

1 public class Robot{
2     private static final int CELL_SIZE= 40;
3     private int xPosition;
4     private int yPosition;
5     private char direction;
6     private boolean isVisible;
7     private Circle body;
8     private Rectangle antenna;
9     private Circle antennaTip;
10    private boolean lastMoveOK;
11

```

- Desarrollen la clase considerando los 3 mini-ciclos. No olviden la documentación. Al final de cada mini-ciclo realicen dos pruebas indicando su propósito. Capturen las pantallas relevantes.

R/=

Ciclo 1

The screenshot shows the BlueJ IDE interface. On the left, the 'robot1 : Robot' instance is displayed with its attributes and values:

Attribute	Value
private int xPosition	2
private int yPosition	3
private char direction	'N'
private boolean isVisible	false
private Circle body	(Visual representation of a circle)
private Rectangle antenna	(Visual representation of a rectangle)
private Circle antennaTip	(Visual representation of a circle)
private boolean lastMoveOK	true

Buttons: Inspeccionar, Obtener, Mostrar campos estáticos, Cerrar.

On the right, the 'Blue: Resultado del Método' window shows the result of the 'robot1.coordinates()' method call:

Retorna la posición lógica actual del robot en el laberinto.
 @return un arreglo de dos posiciones {x, y} con las coordenadas actuales
 int[] coordinates()

robot1.coordinates() retornado:

int[]	Value
[0]	2
[1]	3

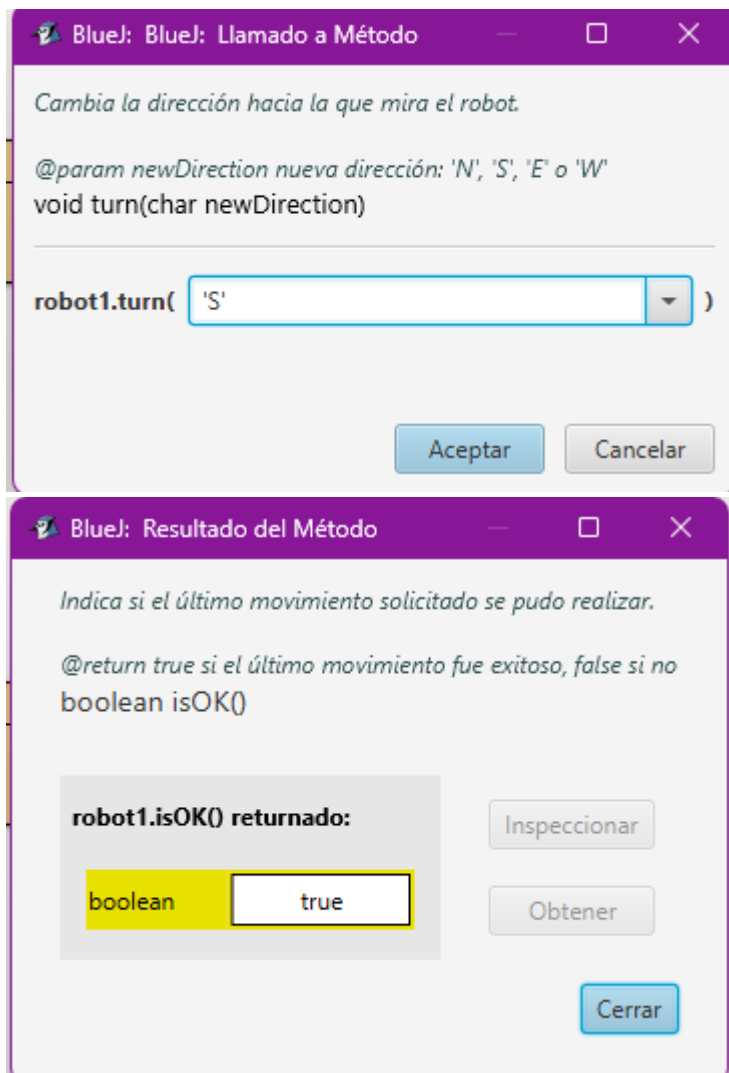
Buttons: Inspeccionar, Obtener, Cerrar.

Below this, another instance of the 'int[]' array is shown with its elements:

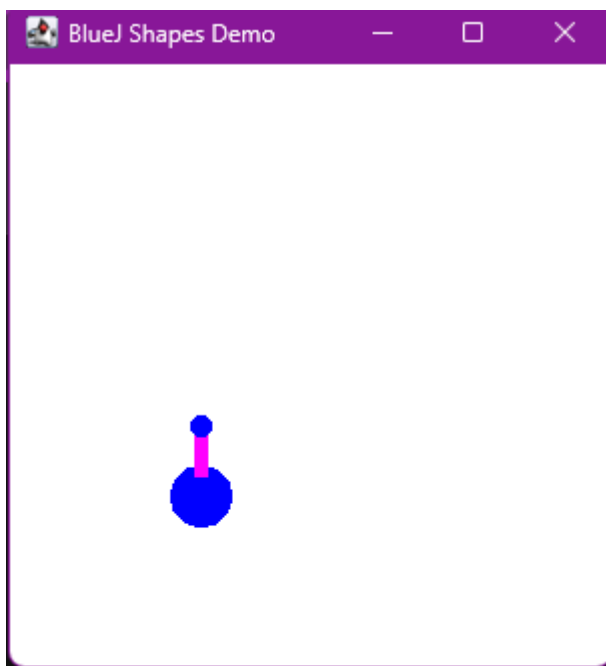
int length	Value
[0]	2
[1]	3

Buttons: Inspeccionar, Obtener, Mostrar campos estáticos, Cerrar.

Ciclo2



Ciclo3



B Definiendo y creando una nueva clase.

1. Diseñen la clase, es decir, definan los métodos que debe ofrecer.

R/=

Requisito funcional: Crear el laberinto indicando su tamaño (ubica entrada/salida)

Método propuesto: RobotMaze(int size) (constructor)

Requisito funcional: Adicionar paredes (solo antes de iniciar)

Método propuesto: void addWall(int x, int y)

Requisito funcional: Iniciar el juego

Método propuesto: void startGame()

Requisito funcional: Hacer los movimientos deseados

Método propuesto: void moveRobot(int steps) y void turnRobot(char direction)

Requisito funcional: Consultar la vida disponible

Método propuesto: int getLives()

Requisito funcional: Indicar condiciones de fin (llegó a la salida / sin vida)

Método propuesto: boolean isGameOver()

Requisito funcional: Terminar el juego cuando se desee

Método propuesto: void endGame()

2. Planifiquen la construcción considerando algunos mini-ciclos.

R/=

Mini-ciclo 1: Creación del laberinto. Se implementa el constructor

RobotMaze(int size), que define el tamaño del tablero y ubica aleatoriamente la entrada y la salida en caras opuestas, como indica el enunciado. Prueba: crear laberintos de distintos tamaños y verificar que entrada y salida queden ubicadas correctamente en caras opuestas.

Mini-ciclo 2: Paredes. Se implementa addWall(int x, int y), permitiendo agregar paredes únicamente antes de iniciar el juego (se valida con un booleano gameStarted). Prueba: agregar paredes válidas y verificar que se rechacen si el juego ya inició, mostrando el mensaje de error correspondiente con JOptionPane.

Mini-ciclo 3: Inicio del juego y movimiento del robot. Se implementa startGame(), que ubica al Robot en la entrada del laberinto con 10 puntos de

vida, y `moveRobot(int steps)` junto con `turnRobot(char direction)`, que delegan en los métodos `move()` y `turn()` de la clase `Robot`. Prueba: iniciar el juego y mover el robot en distintas direcciones, confirmando que su posición cambie correctamente.

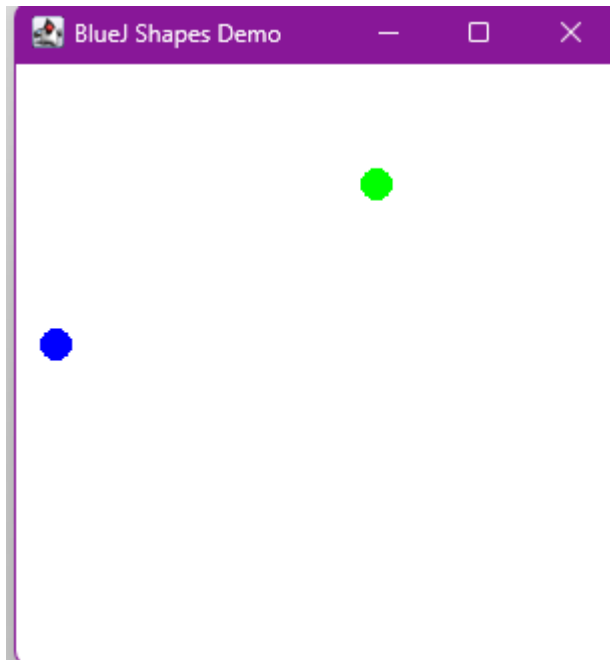
Mini-ciclo 4: Colisiones y vida. Se extiende `moveRobot()` para detectar choques contra paredes o bordes del laberinto, restando un punto de vida y cambiando el color de la pared golpeada. Prueba: mover el robot deliberadamente contra una pared y contra el borde del laberinto, verificando que la vida disminuya y el color cambie en ambos casos.

Mini-ciclo 5: Condiciones de fin de juego. Se implementa `isGameOver()`, que retorna `true` si el robot llegó a la salida o si se quedó sin vida, y `endGame()`, que permite terminar el juego manualmente en cualquier momento. Prueba: forzar cada una de las tres condiciones de fin (llegada a la salida, vida en cero, finalización manual) y confirmar que `isGameOver()` responda correctamente en cada caso.

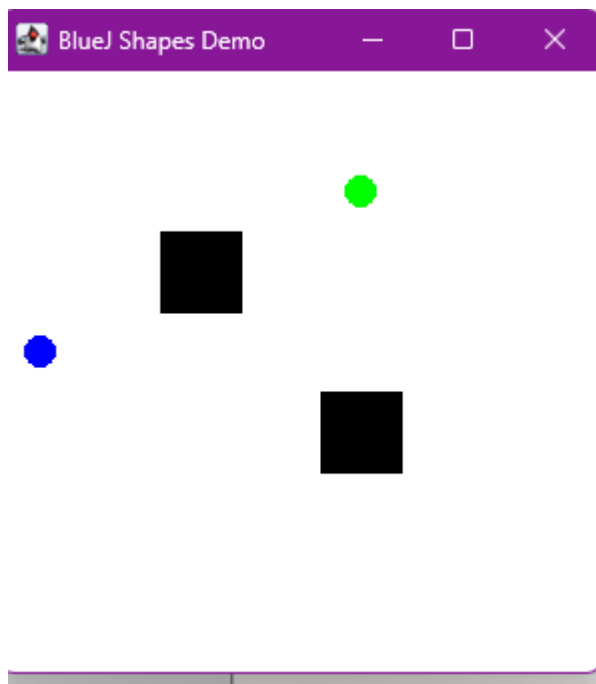
3. Implementen la clase. Al final de cada mini-ciclo realicen una prueba indicando su propósito. Capturen las pantallas relevantes.

R/=

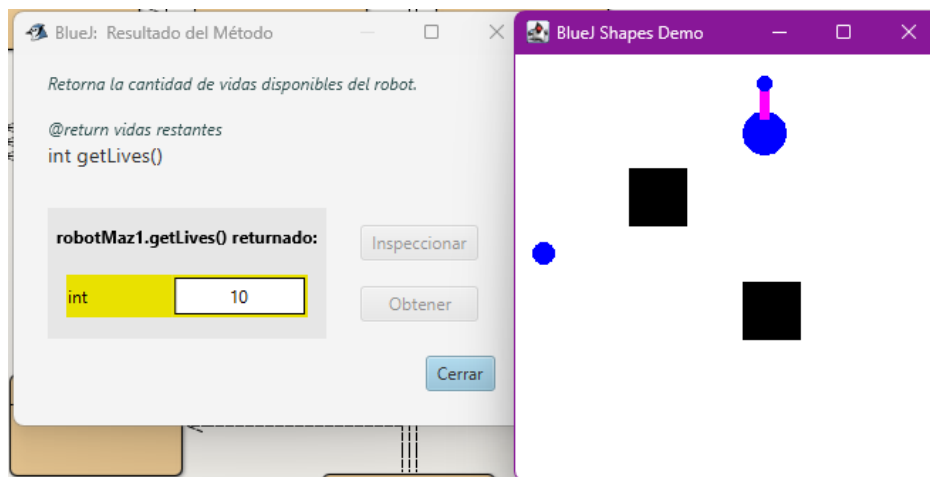
Mini ciclo 1



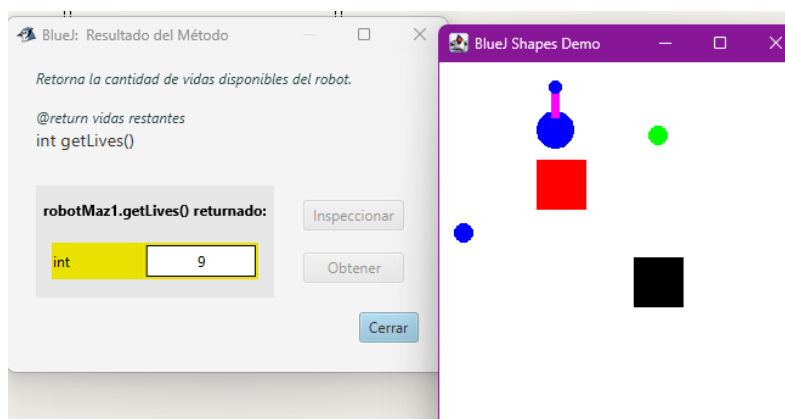
Mini ciclo 2



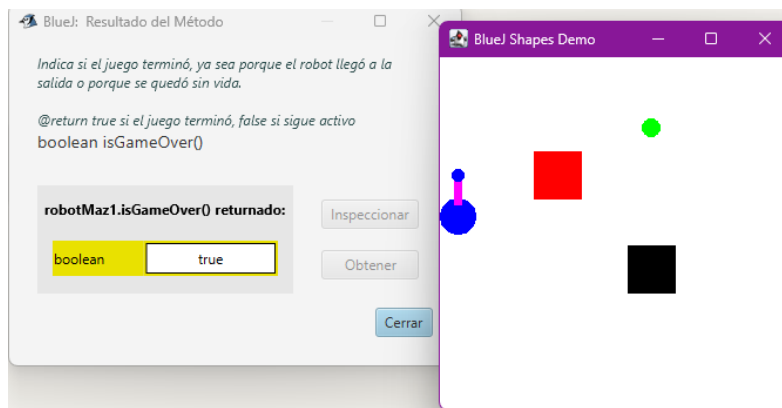
Mini ciclo 3



Mini ciclo 4



Mini ciclo 5



- Indiquen las extensiones necesarias para reutilizar la clase Robot y el paquete shapes. Expliquen.

R/=

Robot se reutiliza sin modificar su código.

RobotMaze gestiona el laberinto (paredes, entrada/salida) y valida los movimientos usando su propia matriz. Si un movimiento es inválido, no llama a move(), resta vida y cambia la pared con shapes.

Para que isOK() refleje choques, RobotMaze debe notificar a Robot mediante un método adicional.

Shapes se usa para dibujar paredes, entrada y salida visualmente.

C BONO. Nuevos requisitos funcionales. RobotMaze.

- Diseñen, es decir, definan los métodos que debe ofrecer.

R/=

Requisito: Hacer un buen movimiento (la máquina decide).

Método propuesto: void autoMove()

Descripción: el robot decide y ejecuta por sí mismo un movimiento hacia una casilla adyacente válida (sin pared y dentro de los límites), sin que el jugador indique dirección ni pasos.

Requisito: Deshacer el último movimiento (contando el deshacer como un movimiento).

Método propuesto: void undo()

Descripción: revierte el efecto del último movimiento realizado (ya sea uno hecho por el jugador con moveRobot() o uno automático con autoMove()), devolviendo al robot a su posición y orientación anteriores. Al ser considerado un movimiento, también puede afectar la vida disponible si el intento de deshacer resulta en un choque.

Estrategia para autoMove() (a justificar en la implementación):

El robot evalúa las 4 direcciones posibles desde su posición actual, descarta las que llevan a una pared o fuera del laberinto, y entre las restantes elige la que lo

acerca más a la salida (comparando la distancia a exitX, exitY antes y después de cada opción). Si ninguna dirección lo acerca, elige cualquiera de las válidas para evitar quedarse inmóvil.

Extensión necesaria en RobotMaze para soportar undo():

Se requiere guardar un historial del estado antes de cada movimiento (posición, dirección y vidas), por ejemplo en una pila (Stack), para poder restaurarlo al llamar undo(). Cada llamada a moveRobot() o autoMove() debe registrar el estado anterior en esa pila antes de ejecutar el movimiento.

2. Implementen los nuevos métodos. Al final de cada método realicen una prueba indicando su propósito. Capturen las pantallas relevantes.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?

(Horas)

R/=

José García: 14h.

Kevin Quezada: 14h

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

R/=

Desarrollado casi en su totalidad, faltaba implementación de que, al cambiar la dirección de movimiento, la antena del robot apunte a esa dirección. Falta el punto 2 del bono.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

4. R/=

Desarrollar por iteraciones, la planificación de los ciclos y mini ciclos nos permitió tener un mejor flujo de trabajo.

5. ¿Cuál consideran fue el mayor logro? ¿Por qué?

R/=

Casi completarlo, la implementación de Robot y RobotMaze es compleja.

6. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

R/=

7. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

8. R/=

Nos comprometemos a seguir manteniendo la calidad en todo el semestre.

9. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

R/=

- OpenAI. (2026). *ChatGPT* [Large language model]. <https://chatgpt.com/>
- Anthropic. (2026). *Claude* [Large language model]. <https://claude.ai/>